Toward Unified Feature Interaction and Sequence Modeling: Improvements on the PCVRHyFormer Baseline for CTR Prediction

1 **2**

/ / /

1 2

Abstract

We address the long-standing gap between feature interaction models and sequential recommendation models in industrial click-through rate (CTR) prediction. Building on the official PCVRHyFormer baseline released for the TAAC 2026 competition, we propose [TODO: describe modifications] and demonstrate improvements over the baseline subject to the official latency constraint. [TODO: add concrete AUC numbers once experiments complete.]

1 Introduction

2 Related Work

2.1 Feature Interaction Models

2.2 Sequential Recommendation Models

2.3 Unified Architectures

3 Problem Formulation and Dataset

3.1 Task Formulation

We formulate click-through rate (CTR) prediction as a binary classification task. Given a user u, a candidate item i, and the user's historical behavior sequences across $D = 4$ heterogeneous domains $S = \{S_a, S_b, S_c, S_d\}$, the model outputs a click probability

$$\hat{y} = f_\theta(u, i, S) \in [0, 1], \tag{1}$$

where f_θ denotes the parameterized model with learnable parameters θ. The model is trained by minimizing the standard binary cross-entropy loss

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^{N} \left[y_n \log \hat{y}_n + (1 - y_n) \log(1 - \hat{y}_n) \right], \tag{2}$$

where $y \in \{0, 1\}$ is the ground-truth label indicating whether the impression resulted in a click, and N is the number of training samples. At inference time, predicted probabilities are used to rank candidate items; the system is evaluated by the area under the ROC curve (AUC), subject to the official latency constraint imposed by the competition organizers.

3.2 Dataset Overview

We use the *TAAC 2026 Academic Track* dataset, which consists of approximately one million impression records derived from real-world Tencent advertising logs. Each record corresponds to a single (user, item, timestamp) impression, augmented with the user's preceding behavior across multiple domains.

The dataset is *fully anonymized*: all sparse features are mapped to integer IDs, all dense features are presented as fixed-length float vectors, and no raw content (text, images, URLs) or personally identifiable information is preserved. As a consequence, *pre-trained language or vision models cannot be leveraged* for feature extraction — methodological contributions must come from architecture design rather than external knowledge transfer.

3.3 Feature Schema

Each record is stored in a *flat schema of 120 columns*, partitioned into six categories (Table 1): five ID/label columns, 46 user integer features (35 scalar + 11 multi-value), 10 user dense features, 14 item integer features, zero item dense features, and 45 domain sequence features distributed across four behavioral domains.

A distinctive feature of the schema is the *aligned (key, value) structure*: eight of the ten user-dense columns are *positionally aligned* with their integer counterparts, encoding per-element behavioral statistics (e.g., dwell time, interaction score) for the corresponding entity IDs.

Table 1: Schema of the TAAC 2026 Academic Track dataset (120 columns).

Category	Count	Type	Description
ID & label	5	`int64 / int32`	Identifiers, click label, timestamp
User integer	46	`int64 / list`	35 scalar + 11 multi-value
User dense	10	`list⟨float⟩`	2 pre-trained + 8 aligned
Item integer	14	`int64 / list`	13 scalar + 1 multi-value
Item dense	0	—	(none)
Domain sequences	45	`list⟨int64⟩`	4 domains: A (9), B (14), C (12), D (10)
Total	**120**		

3.4 Behavioral Domains

The 45 sequence columns are partitioned into four *heterogeneous behavioral domains*, denoted A through D, with column counts 9, 14, 12, and 10 respectively. While the precise business semantics of each domain are not disclosed by the dataset provider, the domains differ in average sequence length and likely correspond to distinct interaction modalities (e.g., feed impressions, search interactions, video views, application usage). To preserve this heterogeneity, our model treats each domain as a separate sequence stream, and any cross-domain reasoning is deferred to the fusion stage rather than performed by collapsing all actions onto a single timeline.

Within each domain, multiple columns are *positionally aligned*: the i-th element across all columns of domain d describes the i-th historical action through complementary attributes such as item identifier, category, and timestamp. Continuous timestamps are not consumed directly; instead, inter-action time deltas are discretized into 65 logarithmically-spaced buckets ranging from 5 seconds to one year, with bucket 0 reserved for padding. Each bucket maps to a learnable embedding, providing the model with a smooth temporal prior that distinguishes recent from distant behavior without exposing raw Unix time. Following the official baseline configuration, sequences are truncated to a maximum length of 256 for domains A and B, and 512 for the longer domains C and D.

4 Methodology

4.1 Baseline Architecture: PCVRHyFormer

4.1.1 Overview

This section describes the official baseline released by the competition organizers, denoted *PCVRHyFormer*. We use this architecture without modification as our starting point; the contributions of this work, presented in Section 4.2, are layered on top of it. Our goal here is to make the baseline self-contained for the reader, so that subsequent comparisons in Section 5 can be interpreted unambiguously.

PCVRHyFormer is a *unified hybrid Transformer* that processes static user/item attributes and temporal behavior sequences within a single architecture, rather than fusing them only at the output layer. The model maintains three semantically distinct sets of tokens — *non-sequential (NS) tokens* derived from static features, *sequence tokens* derived from per-domain behavioral histories, and a small set of learnable *query (Q) tokens* that mediate cross-modal information transfer. These tokens are processed jointly through N stacked *MultiSeqHyFormerBlocks*, after which the final query tokens are pooled and passed to a classifier head producing the click probability $\hat{y}$. Figure 1 illustrates the overall data flow.

Figure 1: Architecture of PCVRHyFormer. **[TODO: replace with actual figure]** The model processes 120-column inputs through three token streams (NS, sequence, Q), interleaved within N MultiSeqHyFormerBlocks.

4.1.2 Embedding Layer

The embedding layer maps raw integer IDs and dense values into a shared d-dimensional vector space, producing the inputs to all subsequent token streams. Three distinct embedding strategies are employed, reflecting the heterogeneity of the 120-column schema.

Sparse feature embeddings. Each integer feature with vocabulary size V is assigned an embedding table $E \in \mathbb{R}^{(V+1)\times d_{\text{emb}}}$, where index 0 is reserved for padding and Xavier-initialized to zero. Multi-value features are mean-pooled across their non-zero entries before being passed downstream. To accommodate features with extremely large cardinalities — for which a full embedding table would exceed practical memory budgets — the baseline applies an *embedding skip threshold*: any feature whose vocabulary size exceeds $V_{\text{skip}} = 10^6$ is replaced with a zero vector and effectively dropped from the model. This is a deliberate engineering trade-off that bounds the parameter footprint at the cost of discarding a small number of long-tail signals.

Dense feature projections. The 10 user-dense columns are concatenated into a single vector and passed through a two-layer block (linear projection + LayerNorm + SiLU) that yields a single d-dimensional NS token. The (key, value)-aligned columns described in Section 3.3 are *not* given special treatment in the baseline: their integer counterparts are embedded independently as sparse features, and their dense counterparts are fused into the same flattened user-dense vector, leaving the positional alignment between the two unexploited at the input stage.

Time-bucket embeddings. Continuous inter-action time deltas are quantized into 65 logarithmically-spaced buckets (Section 3.4) and mapped through a learnable embedding table $E_t \in \mathbb{R}^{65\times d}$, with bucket 0 reserved for padding. Time-bucket vectors are added element-wise to their corresponding sequence tokens, providing a learned temporal prior without exposing raw Unix timestamps to the network.

4.1.3 Token Types

We now describe how the three token streams are constructed from the embedded features. Throughout, let D denote the model dimension ($D = 64$ in the default configuration).

Non-sequential (NS) tokens. NS tokens encode static user and item attributes that do not depend on temporal context. The baseline adopts the *RankMixer*-style tokenizer (**?**): all sparse embeddings within a feature group are concatenated into a single vector, equally split into T_{ns} chunks, and each chunk is independently projected to $\mathbb{R}^D$ via a linear layer with LayerNorm. This produces $T^u_{\text{ns}} = 5$ user NS tokens and $T^i_{\text{ns}} = 2$ item NS tokens. The user-dense vector contributes one additional NS token (Section 4.1.2), giving $M = T^u_{\text{ns}} + 1 + T^i_{\text{ns}} = 8$ NS tokens in total.

Sequence tokens. Each behavioral domain $d \in \{A, B, C, D\}$ is tokenized independently. For position ℓ in domain d, the embeddings of all aligned side-information columns (item ID, category, etc.) are concatenated and projected to $\mathbb{R}^D$, then summed with the corresponding time-bucket embedding. The result is a domain-specific sequence tensor of shape (L_d, D), where $L_d \in \{256, 256, 512, 512\}$ following the truncation policy of Section 3.4. Crucially, the four domains do *not* share a unified embedding stack: each domain maintains its own embedding tables, so no cross-domain ID identification is performed at this stage.

Query (Q) tokens. Q tokens act as learnable *read-out probes* that extract task-relevant signals from the sequences. For each domain, the baseline first constructs a *global context vector* by concatenating the flattened NS tokens with the mean-pooled sequence tokens of that domain, and then transforms this context through N_q independent feed-forward networks to produce N_q query tokens (with $N_q = 2$ in the default configuration). Q tokens are thus *initialized from* both static and behavioral information, but their final values emerge only after multiple rounds of cross-attention and mixing within the block stack (Section 4.1.4).

4.1.4 MultiSeqHyFormerBlock

The core computation of PCVRHyFormer takes place inside a stack of N identical *MultiSeqHyFormerBlocks* (default $N = 2$). Each block consumes the current state of all three token streams and produces refined versions of the same shape, allowing the architecture to be cleanly stacked. Within a single block, four operations are executed in sequence.

(i) Sequence evolution. Each domain d first refines its own sequence tokens through an independent Transformer encoder, equipped with rotary position embeddings (RoPE) (**?**) and a padding mask. Self-attention is restricted to within-domain tokens, so the four domains do not exchange information at this stage. Formally, for each d,

$$S_d \leftarrow \text{SeqEncoder}_d(S_d,\, \text{mask}_d)\,, \tag{3}$$

where $S_d \in \mathbb{R}^{L_d\times D}$ denotes the sequence tokens of domain d.

(ii) Query decoding. The N_q query tokens of each domain then attend over their refined sequence tokens via cross-attention, with the queries acting as the *query* side and the sequence tokens as *key* and *value*. RoPE is applied only on the key/value side. This step lets each query token gather behaviorally relevant context from its assigned domain:

$$Q_d \leftarrow \text{CrossAttn}_d(Q_d,\, S_d)\,, \quad d \in \{A, B, C, D\}. \tag{4}$$

(iii) Token fusion. All decoded query tokens and the shared NS tokens are concatenated into a single token stream of length $T = N_q \cdot |D| + M$ (with default $T = 2 \cdot 4 + 8 = 16$):

$$Z = \text{concat}\big(Q_A, Q_B, Q_C, Q_D, \text{NS}\big) \in \mathbb{R}^{T\times D}. \tag{5}$$

This concatenation is the only point at which static and behavioral information are placed in a common attention context, making it the architectural locus of *deep* static–sequential fusion.

(iv) Query boosting. The fused stream Z is finally processed by a *RankMixer*-style block that performs token mixing and a per-token feed-forward refinement (**?**). Token mixing is parameter-free: each token's D-dimensional representation is reshaped into T sub-spaces of size D/T and the token and sub-space axes are transposed, redistributing information across tokens before a shared FFN refines each token independently. A residual connection and post-LayerNorm yield the boosted output, which is finally split back into per-domain Q_d and NS slots ready to be consumed by the next block. The construction requires $D \bmod T = 0$, a constraint that, together with the chosen number of NS tokens, fixes the admissible values of D in the default configuration.

After the final block, only the per-domain query tokens are retained: they are concatenated, flattened, and projected through a linear–LayerNorm head to produce a single D-dimensional pooled vector, which is then passed to the classifier described in Section 4.1.5.

4.1.5 Output and Training

Classifier head. The pooled D-dimensional output of the block stack is passed through a lightweight classifier consisting of a linear layer, LayerNorm, SiLU activation, dropout, and a final linear projection to a single logit. A sigmoid then yields the predicted click probability $\hat{y}$.

Training configuration. The model is trained end-to-end with the binary cross-entropy loss of Equation (**??**). Following standard practice in industrial recommender systems, parameters are split into two groups optimized by separate routines: all embedding tables ("sparse" parameters) are updated with Adagrad, while the remaining ("dense") parameters are updated with AdamW. This split exploits the fact that embedding gradients are sparse — only a small subset of rows is touched per step — allowing Adagrad's per-coordinate learning rates to outperform a uniform optimizer on these parameters. The default hyperparameters are: dense learning rate 1×10^{-4}, batch size 256, dropout 0.01, $D = 64$, $N_q = 2$, $N = 2$ stacked blocks, and 4 attention heads. Early stopping is applied on the held-out validation AUC with a patience of five evaluation rounds.

4.2 Our Modifications

5 Experiments

5.1 Setup

5.2 Main Results

5.3 Ablation Studies

5.4 Latency Analysis

6 Conclusion

References

[1] Guo, H., Tang, R., Ye, Y., Li, Z. & He, X. (2017) DeepFM: A factorization-machine based neural network for CTR prediction. *IJCAI*.

[2] Kang, W.-C. & McAuley, J. (2018) Self-attentive sequential recommendation. *ICDM*.

[3] Zhou, G., Zhu, X., Song, C., Fan, Y., Zhu, H., Ma, X., Yan, Y., Jin, J., Li, H. & Gai, K. (2018) Deep interest network for click-through rate prediction. *KDD*.

A Technical Appendices and Supplementary Material

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [NA]

   Justification: This paper does not present new theoretical results.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?

   Answer: **[TODO]**

   Justification: **[TODO]**

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code?

   Answer: **[TODO]**

   Justification: **[TODO]**

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details?

   Answer: **[TODO]**

   Justification: **[TODO]**

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined?

   Answer: **[TODO]**

   Justification: **[TODO]**

8. **Experiments compute resources**

   Question: Does the paper provide sufficient information on compute resources?

   Answer: **[TODO]**

   Justification: **[TODO]**

9. **Code of ethics**

   Question: Does the research conducted in the paper conform with the NeurIPS Code of Ethics?

   Answer: [Yes]

   Justification: The research uses an officially anonymized public dataset and introduces no human subjects, surveillance applications, or dual-use risks.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive and negative societal impacts?

    Answer: **[TODO]**

Justification: **[TODO]**

11. **Safeguards**

    Question: Does the paper describe safeguards for responsible release of high-risk data or models?

    Answer: [NA]

    Justification: This paper does not release new models or datasets with high misuse risk.

12. **Licenses for existing assets**

    Question: Are creators or original owners of assets used in the paper properly credited?

    Answer: [Yes]

    Justification: The TAAC 2026 dataset and PCVRHyFormer baseline are properly cited; their use follows the official competition terms.

13. **New assets**

    Question: Are new assets introduced in the paper well documented?

    Answer: [NA]

    Justification: This paper does not release new public assets.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions?

    Answer: [NA]

    Justification: The paper does not involve crowdsourcing or human subjects research.

15. **Institutional review board (IRB) approvals**

    Question: Does the paper describe potential risks incurred by study participants and IRB approvals?

    Answer: [NA]

    Justification: The paper does not involve human subjects research.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important component of the core methods?

    Answer: [NA]

    Justification: LLMs are not used as a core methodological component of this work.